

LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6



GRAPH AND TREE

Oleh:

Nor Khalisyah

NIM.2510817120005

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nor Khalisyah
NIM : 2510817120005

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhammad Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
SOAL 1	1
A. Source Code.....	2
B. Output Program.....	3
C. Pembahasan	4
SOAL 2	5
A. Source Code.....	6
B. Output Program	7
C. Pembahasan	7
SOAL 3	9
A. Source Code.....	10
B. Output Program	12
C. Pembahasan	12
SOAL 4	19
A. Source Code.....	20
B. Output Program	22
C. Pembahasan	22
SOAL 5	14
A. Source Code.....	15
B. Output Program	17
C. Pembahasan	17
TAUTAN GIT.....	24

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	3
Gambar 2. Screenshot Hasil Jawaban Soal 2	7
Gambar 3. Screenshot Hasil Jawaban Soal 3	12
Gambar 4. Screenshot Hasil Jawaban Soal 4	17
Gambar 5. Screenshot Hasil Jawaban Soal 5	22

DAFTAR TABEL

Table 1. Contoh kasus soal 1.....	2
Table 2. Source Code Soal1.cpp	2
Table 3. Contoh kasus soal 2.....	6
Table 4. Source Code Soal 2.cpp	6
Table 5. Contoh kasus soal 3.....	10
Table 6. Source Code Soal 3.cpp	10
Table 7. Contoh kasus soal 4.....	15
Table 8. Source Code Soal 4.cpp	15
Table 9. Contoh kasus soal 10.....	20
Table 10. Source Code Soal 5.cpp	20

SOAL 1

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN M
U1 V1 W1 U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Table 1. Contoh kasus soal 1

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

A. Source Code

Table 2. Source Code Soal1.cpp

1	#include <iostream>
2	#include <vector>
3	#include <string>
4	#include <map>
5	using namespace std;
6	
7	int main() {
8	int N;
9	cin >> N;
10	
11	vector<char> labels(N);
12	map<char, int> idx;
13	for (int i = 0; i < N; i++){

```

14         cin >> labels[i];
15         idx[labels[i]] = i;
16     }
17     vector<vector<int>> matrix(N, vector<int>(N,
0));
18
19     int M;
20     cin >> M;
21     for (int i = 0; i < M; i++){
22         char U, V;
23         int W;
24         cin >> U >> V >> W;
25         matrix[idx[U]][idx[V]] = W;
26     }
27
28     cout << "Adjacency Matrix:" << endl;
29
30     cout << " ";
31     for (int i = 0; i < N; i++) {
32         cout << " " << labels [i];
33     }
34     cout << endl;
35
36     for (int i = 0; i < N; i++) {
37         cout << labels[i];
38         for (int j = 0; j<N; j++){
39             cout << " " << matrix[i][j];
40         }
41         cout << endl;
42     }
43
44     return 0;
45 }

```

B. Output Program



```

PS C:\Users\USER\OneDrive\Documents\Github\module-6-graph-and-tree-norkhalisyah162007-crypto> g++ -o soal1 Soal1.cpp
PS C:\Users\USER\OneDrive\Documents\Github\module-6-graph-and-tree-norkhalisyah162007-crypto> .\soal1.exe
5
A B C D E 6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Masalah yang perlu diselesaikan adalah mengubah representasi graph dari daftar edge menjadi adjacency matrix. `vector<vector<int>> matrix(N, vector<int>(N, 0))` digunakan untuk menyimpan matrix berukuran $N \times N$ karena setiap posisi (i,j) merepresentasikan bobot edge dari vertex i ke vertex j , dan nilai awal 0 sudah merepresentasikan tidak adanya edge tanpa perlu inisialisasi tambahan.

Tantangannya adalah label vertex berupa karakter seperti A, B, C bukan indeks angka sehingga tidak bisa langsung digunakan sebagai indeks matrix. Untuk mengatasi ini digunakan `map<char, int> idx` yang memetakan setiap label ke indeks numeriknya. Ketika label dibaca menggunakan `cin >> labels[i]`, sekaligus dilakukan `idx[labels[i]] = i` untuk menyimpan pemetaan label ke posisi indeksnya. Dengan cara ini ketika edge $U \rightarrow V$ W dibaca, posisi `matrix[idx[U]][idx[V]]` langsung bisa diisi dengan W tanpa perlu mencari posisi label secara manual di setiap edge.

Hanya satu posisi yang diisi per edge karena graph pada soal ini bersifat berarah, edge $U \rightarrow V$ tidak berarti ada edge $V \rightarrow U$, sehingga mengisi (j,i) juga akan menghasilkan output yang salah.

Untuk pencetakan, pertama dicetak baris header kolom dengan iterasi `for (int i = 0; i < N; i++) cout << " " << labels[i]` agar nama vertex muncul sebagai judul kolom. Kemudian untuk setiap baris i , dicetak `labels[i]` sebagai label baris diikuti seluruh nilai `matrix[i][j]` dengan iterasi `for (int j = 0; j < N; j++) cout << " " << matrix[i][j]`. Urutan pencetakan mengikuti urutan input karena label disimpan di `vector<char> labels` yang diisi sesuai urutan pembacaan.

Analisis Kompleksitas Waktu dan Ruang: Pembacaan label vertex berjalan $O(N)$ dan pembacaan edge berjalan $O(M \log N)$ karena setiap pencarian indeks di `map` membutuhkan $O(\log N)$. Pencetakan matrix berjalan $O(N^2)$ karena seluruh $N \times N$ posisi dicetak. Kompleksitas waktu keseluruhan adalah $O(N^2 + M \log N)$. Kompleksitas ruang adalah $O(N^2)$ untuk matrix dan $O(N)$ untuk menyimpan label serta pemetaan indeks.

SOAL 2

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

```
V1 -> (X1,w1), (X2,w2), ...
V2 -> (X1,w1), (X2,w2), ...
...
VN -> ...
```

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Table 3. Contoh kasus soal 2

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4) , (C,2) B -> (D,7) C -> (B,1) , (E,6) D -> (C,3) E -> -

A. Source Code

Table 4. Source Code Soal 2.cpp

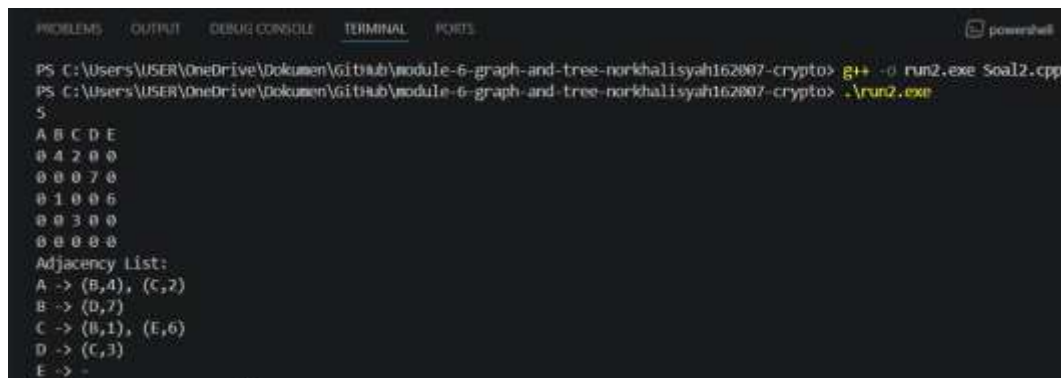
1	#include <iostream>
2	#include <vector>
3	#include <string>
4	using namespace std;
5	
6	int main() {
7	int N;
8	cin >> N;
9	
10	vector<char> labels(N);
11	for (int i = 0; i < N; i++) cin >> labels[i];
12	
13	vector<vector<int>> matrix(N, vector<int>(N));
14	for (int i = 0; i < N; i++)
15	for (int j = 0; j < N; j++)
16	cin >> matrix[i][j];
17	
18	cout << "Adjacency List:" << endl;
19	for (int i = 0; i < N; i++) {

```

20         cout << labels[i] << " -> ";
21         bool hasEdge = false;
22         bool first = true;
23         for (int j = 0; j < N; j++) {
24             if (matrix[i][j] > 0) {
25                 if (!first) cout << ", ";
26                 cout << "(" << labels[j] << ", " <<
matrix[i][j] << ")";
27                 first = false;
28                 hasEdge = true;
29             }
30         }
31         if (!hasEdge) cout << "-";
32         cout << endl;
33     }
34     return 0;
35 }

```

B. Output Program



```

PS C:\Users\USER\OneDrive\Documents\github\module-6-graph-and-tree-norkhalisyah162807-crypto> g++ -o run2.exe Soal2.cpp
PS C:\Users\USER\OneDrive\Documents\github\module-6-graph-and-tree-norkhalisyah162807-crypto> .\run2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -

```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Masalah yang perlu diselesaikan adalah mengubah representasi graph dari adjacency matrix menjadi adjacency list. Adjacency list dicetak dengan menelusuri setiap baris matrix, untuk vertex i , setiap kolom j yang bernilai lebih dari 0 berarti ada edge dari vertex i ke vertex j dengan bobot sebesar nilai tersebut.

`vector<char> labels(N)` digunakan untuk menyimpan label vertex sesuai urutan input karena urutan pencetakan adjacency list harus mengikuti urutan label pada input. Kemudian `vector<vector<int>> matrix(N, vector<int>(N))` digunakan untuk menyimpan seluruh nilai matrix yang

dibaca dari input menggunakan dua perulangan bersarang `for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) cin >> matrix[i][j]`.

Untuk mencetak adjacency list, setiap vertex i ditelusuri menggunakan perulangan luar. Di dalam perulangan dalam, setiap kolom j dicek menggunakan `if (matrix[i][j] > 0)`, jika nilainya lebih dari 0 berarti ada edge dan dicetak dalam format `(labels[j], matrix[i][j])`. Variabel `bool first` digunakan untuk mengontrol pemisah koma agar koma hanya dicetak di antara dua entry bukan sebelum entry pertama. Variabel `bool hasEdge` digunakan untuk mendeteksi apakah vertex i memiliki edge keluar sama sekali jika tidak ada edge yang ditemukan setelah seluruh kolom diperiksa, dicetak sesuai format output.

Analisis Kompleksitas Waktu dan Ruang: Pembacaan matrix berjalan $O(N^2)$ karena seluruh $N \times N$ nilai dibaca. Pencetakan adjacency list juga berjalan $O(N^2)$ karena untuk setiap vertex, seluruh kolom diperiksa satu per satu. Kompleksitas waktu keseluruhan adalah $O(N^2)$. Kompleksitas ruang adalah $O(N^2)$ untuk menyimpan matrix dan $O(N)$ untuk menyimpan label vertex.

SOAL 3

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

0 menyatakan jalan yang dapat dilewati.

1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ...
GRC SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.

- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Table 5. Contoh kasus soal 3

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	<pre> 16 </pre>

A. Source Code

Table 6. Source Code Soal 3.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <queue></code>
4	<code>using namespace std;</code>
5	
6	<code>int main() {</code>
7	<code> int R, C;</code>
8	<code> cin >> R >> C;</code>
9	
10	<code> vector<vector<int>> grid(R, vector<int>(C));</code>

```

11     for (int i = 0; i < R; i++)
12         for (int j = 0; j < C; j++)
13             cin >> grid[i][j];
14
15     int SR, SC, FR, FC;
16     cin >> SR >> SC >> FR >> FC;
17
18     vector<vector<int>> dist(R, vector<int>(C, -
19 1));
20     queue<pair<int,int>> q;
21     dist[SR][SC] = 0;
22     q.push({SR, SC});
23
24     int dr[] = {-1, 1, 0, 0};
25     int dc[] = {0, 0, -1, 1};
26
27     while (!q.empty()) {
28         auto p = q.front();
29         q.pop();
30         int r = p.first;
31         int c = p.second;
32
33         if (r == FR && c == FC) break;
34
35         for (int d = 0; d < 4; d++) {
36             int nr = r + dr[d];
37             int nc = c + dc[d];
38             if (nr >= 0 && nr < R && nc >= 0 && nc
39 < C &&
40 grid[nr][nc] == 0 && dist[nr][nc]
41 == -1) {
42                 dist[nr][nc] = dist[r][c] + 1;
43                 q.push({nr, nc});
44             }
45         }
46     }
47     cout << dist[FR][FC] << endl;
48     return 0;
49 }

```


B. Output Program



```
PS C:\Users\USER\OneDrive\Dokumen\Github\Module-6-graph-and-tree-norkhalisyah162807-crypto> g++ -o soal3 Soal3.cpp
PS C:\Users\USER\OneDrive\Dokumen\Github\Module-6-graph-and-tree-norkhalisyah162807-crypto> .\soal3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 0 0 0 1 0 0 0
0 0
7 9
16
```

Gambar 3. Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Masalah yang perlu diselesaikan adalah mencari jumlah langkah minimum dari posisi awal ke posisi tujuan pada sebuah grid labirin. BFS dipilih karena BFS menjamin jalur yang ditemukan pertama kali adalah jalur terpendek, setiap level BFS merepresentasikan jarak satu langkah lebih jauh dari posisi awal, sehingga ketika posisi tujuan pertama kali ditemukan, jarak yang tercatat sudah pasti merupakan jarak minimum.

`vector<vector<int>> dist(R, vector<int>(C, -1))` digunakan untuk menyimpan jarak dari posisi awal ke setiap sel sekaligus berfungsi sebagai penanda apakah sel sudah pernah dikunjungi, nilai -1 berarti sel belum dikunjungi. Ini menghindari kebutuhan array `visited` terpisah karena kedua informasi digabung dalam satu struktur.

`queue<pair<int,int>> q` digunakan untuk menyimpan sel-sel yang akan diproses. Posisi awal langsung dimasukkan ke queue dengan `dist[SR][SC] = 0` dan `q.push({SR, SC})`. Setiap iterasi mengambil sel terdepan dari queue, lalu memeriksa empat tetangganya menggunakan array arah `dr[]` dan `dc[]`. Tetangga yang valid, yaitu yang berada di dalam batas grid, bernilai 0, dan belum dikunjungi, diberi jarak `dist[r][c] + 1` dan dimasukkan ke queue. Loop berhenti lebih awal dengan `if (r == FR && c == FC) break` ketika posisi tujuan ditemukan karena tidak perlu memproses sel lain setelah tujuan tercapai.

Jika posisi tujuan tidak bisa dicapai, nilai `dist[FR][FC]` tetap -1 dan langsung dicetak sebagai output sesuai format soal.

Analisis Kompleksitas Waktu dan Ruang: Setiap sel pada grid dikunjungi maksimal satu kali karena sel yang sudah memiliki nilai dist tidak akan dimasukkan ke queue lagi. Kompleksitas waktu adalah $O(R \times C)$ karena dalam kasus terburuk seluruh sel grid diproses. Kompleksitas ruang adalah $O(R \times C)$ untuk menyimpan grid, array dist, dan queue yang dalam kasus terburuk bisa menampung seluruh sel grid.

SOAL 4

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Table 7. Contoh kasus soal 4

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

A. Source Code

Table 8. Source Code Soal 4.cpp

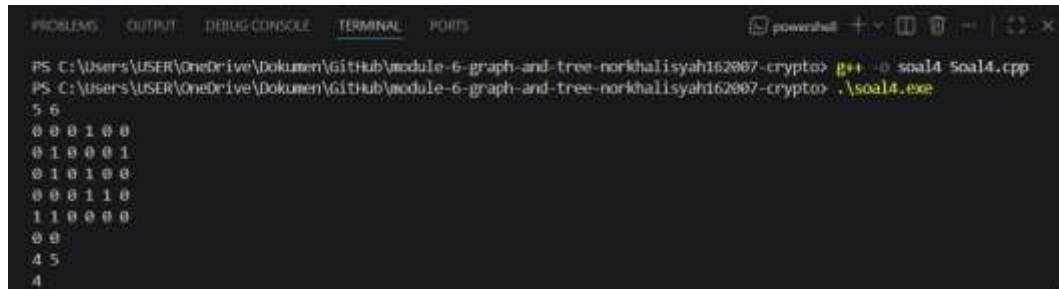
1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int R, C, FR, FC;
6	vector<vector<int>> grid;
7	vector<vector<bool>> visited;
8	int pathCount = 0;
9	
10	int dr[] = {-1, 1, 0, 0};
11	int dc[] = {0, 0, -1, 1};
12	
13	void dfs(int r, int c) {

```

14     if (r == FR && c == FC) {
15         pathCount++;
16         return;
17     }
18
19     for (int d = 0; d < 4; d++) {
20         int nr = r + dr[d];
21         int nc = c + dc[d];
22         if (nr >= 0 && nr < R && nc >= 0 && nc < C
23     &&
24         grid[nr][nc] == 0 && !visited[nr][nc])
25     {
26         visited[nr][nc] = true;
27         dfs(nr, nc);
28         visited[nr][nc] = false; // backtrack
29     }
30 }
31
32 int main() {
33     cin >> R >> C;
34     grid.assign(R, vector<int>(C));
35     visited.assign(R, vector<bool>(C, false));
36
37     for (int i = 0; i < R; i++)
38         for (int j = 0; j < C; j++)
39             cin >> grid[i][j];
40
41     int SR, SC;
42     cin >> SR >> SC >> FR >> FC;
43
44     visited[SR][SC] = true;
45     dfs(SR, SC);
46
47     cout << pathCount << endl;
48     return 0;

```

B. Output Program



```
PS C:\Users\USER\OneDrive\Documents\Github\module-6-graph-and-tree-norkhalisyah162887-crypto> g++ -o soal4 Soal4.cpp
PS C:\Users\USER\OneDrive\Documents\Github\module-6-graph-and-tree-norkhalisyah162887-crypto> .\soal4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Masalah yang perlu diselesaikan adalah menghitung banyaknya jalur sederhana dari posisi awal ke posisi tujuan pada grid labirin. DFS rekursif dengan backtracking dipilih karena tujuannya bukan mencari satu jalur terpendek melainkan menghitung semua jalur yang mungkin, BFS tidak cocok untuk ini karena BFS tidak menelusuri semua kemungkinan percabangan secara mendalam.

`vector<vector<bool>> visited(R, vector<bool>(C, false))` digunakan untuk menandai sel yang sudah dikunjungi dalam jalur saat ini agar jalur yang sama tidak melewati sel yang sama lebih dari satu kali. `int pathCount = 0` digunakan sebagai penghitung jalur yang berhasil mencapai posisi tujuan, dideklarasikan sebagai variabel global agar bisa diakses dan diperbarui langsung dari dalam fungsi `dfs()` tanpa perlu dioper sebagai parameter.

Fungsi `dfs(int r, int c)` bekerja rekursif. Ketika posisi `(r,c)` sama dengan posisi tujuan `(FR,FC)`, `pathCount++` dicatat dan fungsi langsung return karena jalur ini sudah selesai. Jika belum sampai tujuan, fungsi memeriksa empat tetangga menggunakan array arah `dr[]` dan `dc[]`. Tetangga yang valid, berada di dalam batas grid, bernilai 0, dan belum dikunjungi, ditandai `visited[nr][nc] = true` sebelum DFS dilanjutkan ke tetangga tersebut. Setelah DFS dari tetangga selesai, `visited[nr][nc] = false` dijalankan untuk membatalkan tanda kunjungan, inilah mekanisme backtracking yang memungkinkan sel yang sama bisa dikunjungi lagi di jalur yang berbeda.

Di `main()`, posisi awal ditandai `visited[SR][SC] = true` sebelum `dfs(SR, SC)` dipanggil karena posisi awal adalah bagian dari jalur dan tidak boleh dikunjungi dua kali dalam jalur yang sama.

Analisis Kompleksitas Waktu dan Ruang: Kompleksitas waktu DFS dengan backtracking pada grid adalah $O(4^{(R \times C)})$ pada kasus terburuk karena setiap sel bisa memiliki empat pilihan arah dan seluruh kemungkinan jalur dieksplorasi. Pada praktiknya jauh lebih cepat karena banyak cabang dipotong oleh kondisi validitas dan pengecekan `visited`. Kompleksitas ruang adalah $O(R \times C)$ untuk array `visited` dan $O(R \times C)$ untuk kedalaman rekursi maksimum pada call stack.

SOAL 5

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree. Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.

INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.

POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.

ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N Q$

$T_1 T_2$

$\dots T_Q$

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder]: daftar_angka [Inorder]: daftar_angka [Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$

- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Table 9. Contoh kasus soal 10

Input	Output
7	[Preorder]: 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder]: 20 40 30 60 80 70 50
PREORDER	[Preorder]: 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder]: 20 40 30 60 80 70 50
ALL	

A. Source Code

Table 10. Source Code Soal 5.cpp

1	#include <iostream>
2	#include <sstream>
3	#include <string>
4	#include "RedBlackTree.h"
5	using namespace std;
6	
7	void preorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil,
8	string& out, bool& first) {
9	if (node->isNil) return;
10	if (!first) out += " ";
11	out += to_string(node->key);
12	first = false;
13	preorder(node->left, nil, out, first);
14	preorder(node->right, nil, out, first);
15	}
16	
17	void inorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil,
18	string& out, bool& first) {
19	if (node->isNil) return;
20	inorder(node->left, nil, out, first);
21	if (!first) out += " ";
22	out += to_string(node->key);
23	first = false;
24	inorder(node->right, nil, out, first);
25	}
26	
27	

```

28 void postorder(const RedBlackTree::Node* node,
29 const RedBlackTree::Node* nil,
30 string& out, bool& first) {
31     if (node->isNil) return;
32     postorder(node->left, nil, out, first);
33     postorder(node->right, nil, out, first);
34     if (!first) out += " ";
35     out += to_string(node->key);
36     first = false;
37 }
38
39 string getPreorder(const RedBlackTree& rbt) {
40     string out; bool first = true;
41     preorder(rbt.root(), rbt.nil(), out, first);
42     return out;
43 }
44
45 string getInorder(const RedBlackTree& rbt) {
46     string out; bool first = true;
47     inorder(rbt.root(), rbt.nil(), out, first);
48     return out;
49 }
50
51 string getPostorder(const RedBlackTree& rbt) {
52     string out; bool first = true;
53     postorder(rbt.root(), rbt.nil(), out, first);
54     return out;
55 }
56
57 int main() {
58     int N;
59     cin >> N;
60
61     RedBlackTree rbt;
62     for (int i = 0; i < N; i++) {
63         int val;
64         cin >> val;
65         if (!rbt.contains(val)) {
66             rbt.insert(val);
67         }
68     }
69
70     int Q;
71     cin >> Q;
72
73     while (Q--) {
74         string query;
75         cin >> query;

```

```

76         if (rbt.empty()) {
77             cout << "Tree kosong. Tidak ada yang
bisa ditampilkan." << endl;
             continue;
78         }
79
80         if (query == "PREORDER") {
81             cout << "[Preorder] : " <<
getPreorder(rbt) << endl;
82         } else if (query == "INORDER") {
83             cout << "[Inorder] : " <<
getInorder(rbt) << endl;
84         } else if (query == "POSTORDER") {
85             cout << "[Postorder] : " <<
getPostorder(rbt) << endl;
86         } else if (query == "ALL") {
87             cout << "[Preorder] : " <<
getPreorder(rbt) << endl;
88             cout << "[Inorder] : " <<
getInorder(rbt) << endl;
89             cout << "[Postorder] : " <<
getPostorder(rbt) << endl;
90         }
91     }
92     return 0;
93 }

```

B. Output Program

```

PS C:\Users\USER\OneDrive\Dokumen\GitHub\module-6-graph-and-tree-norkhalisyah162007-crypto> g++ -o run5.exe Soal5.cpp RedBlackTree.cpp
PS C:\Users\USER\OneDrive\Dokumen\GitHub\module-6-graph-and-tree-norkhalisyah162007-crypto> .\run5.exe
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Masalah yang perlu diselesaikan adalah memasukkan N bilangan ke dalam Red-Black Tree lalu menjawab query traversal. Red-Black Tree digunakan karena

strukturnya sebagai self-balancing BST menjamin operasi insert dan pencarian selalu berjalan dalam $O(\log N)$ tanpa perlu penyeimbangan manual.

Duplikat ditangani dengan mengecek `if (!rbt.contains(val))` sebelum `rbt.insert(val)` — jika nilai sudah ada di tree maka tidak dimasukkan kembali. Pengecekan ini dilakukan di luar struktur tree agar logika penolakan duplikat eksplisit terlihat di kode utama.

Tiga fungsi traversal `preorder()`, `inorder()`, dan `postorder()` diimplementasikan secara rekursif dengan parameter `string& out` dan `bool& first` yang dioper by reference. `string& out` digunakan untuk menggabungkan hasil traversal secara bertahap tanpa membuat string baru di setiap node. `bool& first` digunakan untuk mengontrol spasi — spasi hanya dicetak sebelum elemen kedua dan seterusnya, bukan sebelum elemen pertama. Kondisi berhenti rekursi adalah `if (node->isNil) return` karena Red-Black Tree menggunakan sentinel node `nil` sebagai penanda daun, bukan `nullptr`.

Fungsi `getPreorder()`, `getInorder()`, dan `getPostorder()` berfungsi sebagai pembungkus yang menginisialisasi `out` dan `first` sebelum memanggil fungsi traversal rekursif, sehingga pemanggilan dari `main()` bisa dilakukan dalam satu baris.

Di `main()`, setiap query dibaca menggunakan `cin >> query` lalu dicocokkan dengan kondisi `if-else`. Jika query adalah `PREORDER`, `INORDER`, atau `POSTORDER` maka hanya satu traversal yang dicetak. Jika query adalah `ALL` maka ketiga traversal dicetak berurutan. Jika tree kosong, dicetak pesan "Tree kosong. Tidak ada yang bisa ditampilkan." tanpa menjalankan traversal apapun.

Analisis Kompleksitas Waktu dan Ruang: Proses insert N bilangan ke Red-Black Tree berjalan $O(N \log N)$ karena setiap insert membutuhkan $O(\log N)$ dan pengecekan `contains()` juga $O(\log N)$. Setiap traversal berjalan $O(N)$ karena setiap node dikunjungi tepat satu kali. Untuk Q query, total kompleksitas traversal adalah $O(Q \times N)$. Kompleksitas waktu keseluruhan adalah $O(N \log N + Q \times N)$. Kompleksitas ruang adalah $O(N)$ untuk menyimpan node-node di dalam Red-Black Tree dan $O(N)$ untuk string hasil traversal.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-norkhalisyah162007-crypto.git>